Plus courts chemins dans les graphes pondérés

Quentin Fortier

Graphe pondéré : Poids

On considère dans ce cours seulement des graphes orientés.

Définition

Un graphe **pondéré** est un graphe $\vec{G} = (V, \vec{E})$ muni d'une fonction de poids $w : \vec{E} \longrightarrow \mathbb{R}$.
$w(u, v)$ est le **poids** de l'arête de u vers v.

Graphe pondéré : Poids

On considère dans ce cours seulement des graphes orientés.

Définition

Un graphe **pondéré** est un graphe $\vec{G} = (V, \vec{E})$ muni d'une fonction de poids $w : \vec{E} \longrightarrow \mathbb{R}$.
$w(u, v)$ est le **poids** de l'arête de u vers v.

Il est pratique de définir $w(u, v) = \infty$ s'il n'y a pas d'arête entre u et v.

Pour représenter un graphe pondéré, on utilisera une **matrice d'adjacence pondéré**, contenant $w(u, v)$ sur la ligne u, colonne v.

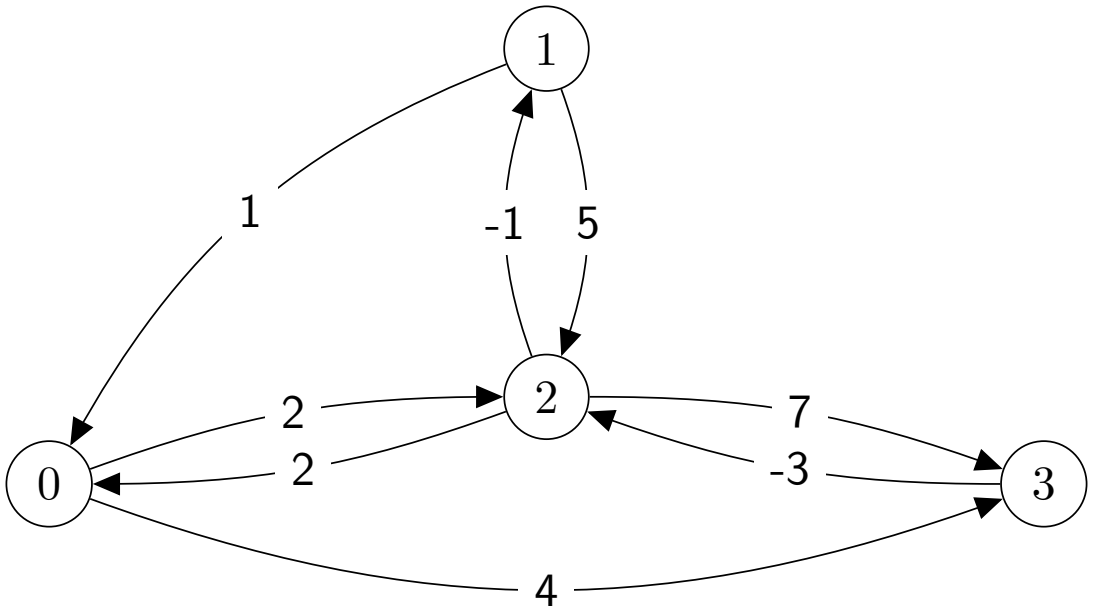
Graphe pondéré : Poids

Il est pratique de dire $w(u, v) = \infty$ s'il n'y a pas d'arête entre u et v. En OCaml, on utilise `max_int`. Mais on aura besoin de gérer les dépassements :

```
let sum x y =
  if x = max_int || y = max_int then max_int
  else x + y
```

Graphe pondéré : Poids

Exemple de graphe représenté par matrice d'adjacence pondérée :



$$\begin{pmatrix} 0 & \infty & 2 & 4 \\ 1 & 0 & 5 & \infty \\ 2 & -1 & 0 & 7 \\ \infty & \infty & -3 & 0 \end{pmatrix}$$

```
let g = [|
    [|0; max_int; 2; 4|],
    [|1; 0; 5; max_int|],
    [|2; -1; 0; 7|],
    [|max_int; max_int; -3; 0|]
|]
```

Graphe pondéré : Chemin

Définition

1. Le **poids d'un chemin** C, noté $w(C)$ est la somme des poids de ses arêtes.
2. Un chemin de $u \in V$ à $v \in V$ est un **plus court chemin** s'il n'existe pas de chemin de poids plus petit.

Graphe pondéré : Chemin

Définition

1. Le **poids d'un chemin** C, noté $w(C)$ est la somme des poids de ses arêtes.
2. Un chemin de $u \in V$ à $v \in V$ est un **plus court chemin** s'il n'existe pas de chemin de poids plus petit.

Exercice

Écrire une fonction `poids_chemin c g` qui calcule le poids d'un chemin `c` (donné par la liste des ses sommets) dans le graphe représenté par la matrice d'adjacence pondérée `g`.

Graphe pondéré : Distance

Définition

La **distance** $d(u, v)$ est le poids d'un plus court chemin de u à v.
Autrement dit :

$$d(u, v) = \inf\{w(C) \mid C \text{ est un chemin de } u \text{ à } v\}$$

Graphe pondéré : Distance

Définition

La **distance** $d(u, v)$ est le poids d'un plus court chemin de u à v.
Autrement dit :

$$d(u, v) = \inf\{w(C) \mid C \text{ est un chemin de } u \text{ à } v\}$$

Il peut ne pas y avoir de plus court chemin de u à v...

Graphe pondéré : Distance

Définition

La **distance** $d(u, v)$ est le poids d'un plus court chemin de u à v.
Autrement dit :

$$d(u, v) = \inf\{w(C) \mid C \text{ est un chemin de } u \text{ à } v\}$$

Il peut ne pas y avoir de plus court chemin de u à v...

- ... si v n'est pas atteignable depuis u : on pose $d(u, v) = \infty$.

Graphe pondéré : Distance

Définition

La **distance** $d(u, v)$ est le poids d'un plus court chemin de u à v.
Autrement dit :

$$d(u, v) = \inf\{w(C) \mid C \text{ est un chemin de } u \text{ à } v\}$$

Il peut ne pas y avoir de plus court chemin de u à v...

- ... si v n'est pas atteignable depuis u : on pose $d(u, v) = \infty$.
- ... s'il existe un cycle de poids négatif : on pose $d(u, v) = -\infty$.

Distance

Il peut ne pas y avoir de plus court chemin de u à v...

Distance

Il peut ne pas y avoir de plus court chemin de u à v...

1. ... si v n'est pas atteignable depuis u, on pose $d(u, v) = \infty$.

Distance

Il peut ne pas y avoir de plus court chemin de u à v...

1. ... si v n'est pas atteignable depuis u, on pose $d(u, v) = \infty$.
2. ... s'il existe un cycle de poids négatif, on pose $d(u, v) = -\infty$.

Remarque : s'il n'y a pas de cycle de poids ≤ 0, les plus courts chemins sont élémentaires (ils passent au plus une fois sur un sommet) donc sont de longueurs au plus $n - 1$.

Propriétés

Inégalité triangulaire

S'il n'y a pas de cycle de poids négatif :

$$\boxed{d(v_1, v_2) \leq d(v_1, v_3) + d(v_3, v_2)}$$

Propriétés

Inégalité triangulaire

S'il n'y a pas de cycle de poids négatif :

$$\boxed{d(v_1, v_2) \leq d(v_1, v_3) + d(v_3, v_2)}$$

Sous-optimalité

Soit C un plus court chemin de u à v et u', v' deux sommets de C. Alors le sous-chemin de C de u' à v' est aussi un plus court chemin.

Propriétés

Inégalité triangulaire

S'il n'y a pas de cycle de poids négatif :

$$\boxed{d(v_1, v_2) \leq d(v_1, v_3) + d(v_3, v_2)}$$

Sous-optimalité

Soit C un plus court chemin de u à v et u', v' deux sommets de C. Alors le sous-chemin de C de u' à v' est aussi un plus court chemin.

<u>Preuve</u> : si ce n'était pas le cas on pourrait le remplacer par un chemin plus court pour obtenir un chemin de u à v plus court que C : absurde.

Sous-optimalité

L'optimalité des sous-problèmes est cruciale pour montrer que certains algorithmes/raisonnements sont corrects, par exemple :

1. pour utiliser la programmation dynamique / diviser pour régner.
2. un sous-arbre d'un ABR (optimal) est un ABR (optimal).
3. un sous-arbre d'un tas est un tas.

Pensez à le mentionner et le justifier si besoin est...

Plus courts chemins

L'objectif du cours est de résoudre le problème suivant :

Problème

Entrée : $\vec{G} = (V, \vec{E})$ un graphe pondéré orienté et $s \in V$.
Sortie : Une liste contenant $d(s, v)$, pour tout $v \in V$.

Plus courts chemins

L'objectif du cours est de résoudre le problème suivant :

Problème

Entrée : $\vec{G} = (V, \vec{E})$ un graphe pondéré orienté et $s \in V$.
Sortie : Une liste contenant $d(s, v)$, pour tout $v \in V$.

Cas particuliers :

- Tous les poids sont égaux :

Plus courts chemins

L'objectif du cours est de résoudre le problème suivant :

Problème

Entrée : $\vec{G} = (V, \vec{E})$ un graphe pondéré orienté et $s \in V$.
Sortie : Une liste contenant $d(s, v)$, pour tout $v \in V$.

Cas particuliers :

- Tous les poids sont égaux : **parcours en largeur** depuis s.

Plus courts chemins

L'objectif du cours est de résoudre le problème suivant :

Problème

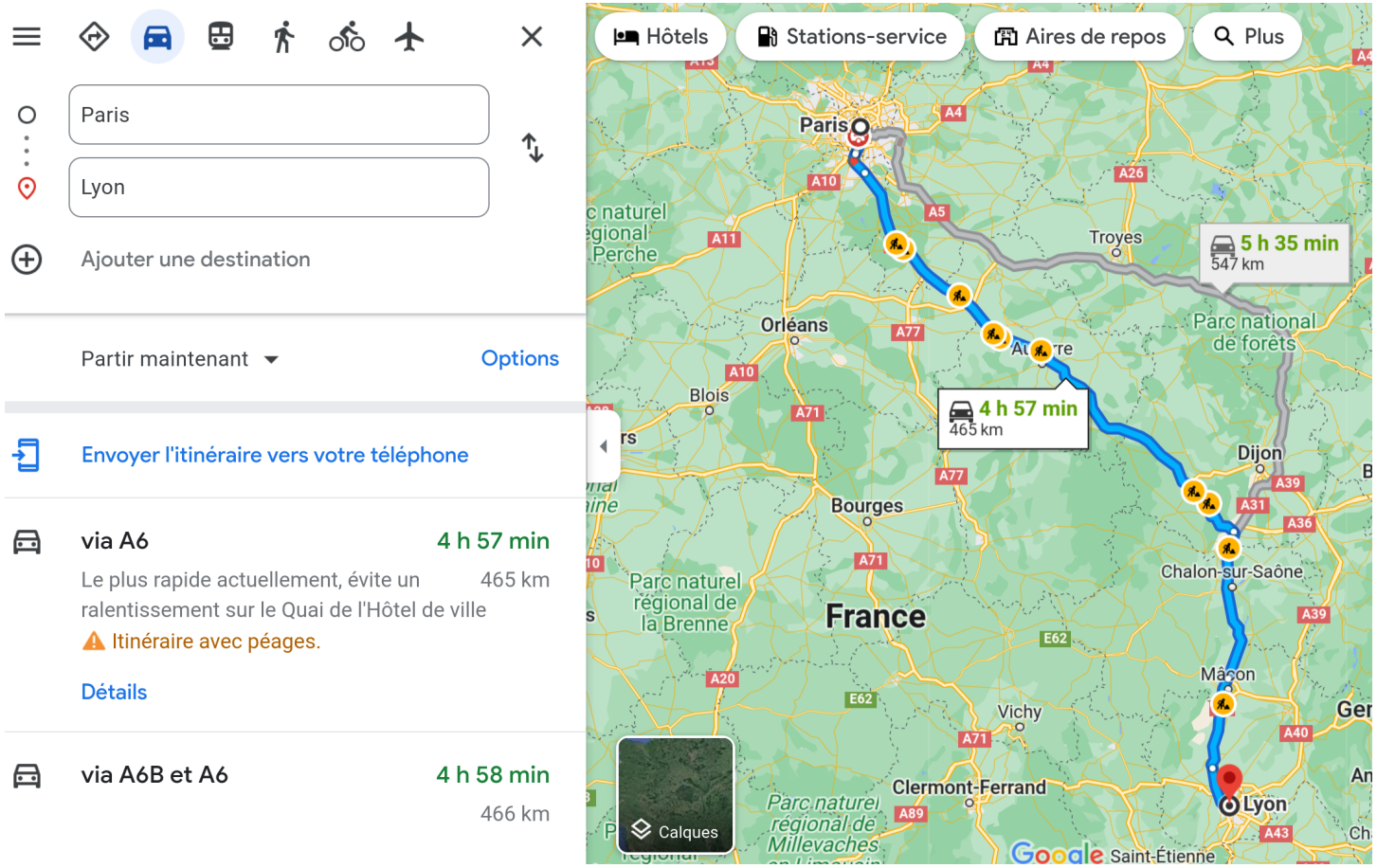
Entrée : $\vec{G} = (V, \vec{E})$ un graphe pondéré orienté et $s \in V$.
Sortie : Une liste contenant $d(s, v)$, pour tout $v \in V$.

Cas particuliers :

- Tous les poids sont égaux : **parcours en largeur** depuis s.
- Tous les poids sont positifs : **algorithme de Dijkstra** depuis s.

Plus courts chemins

Exemple d'application : trouver le plus court chemin d'une ville à une autre.



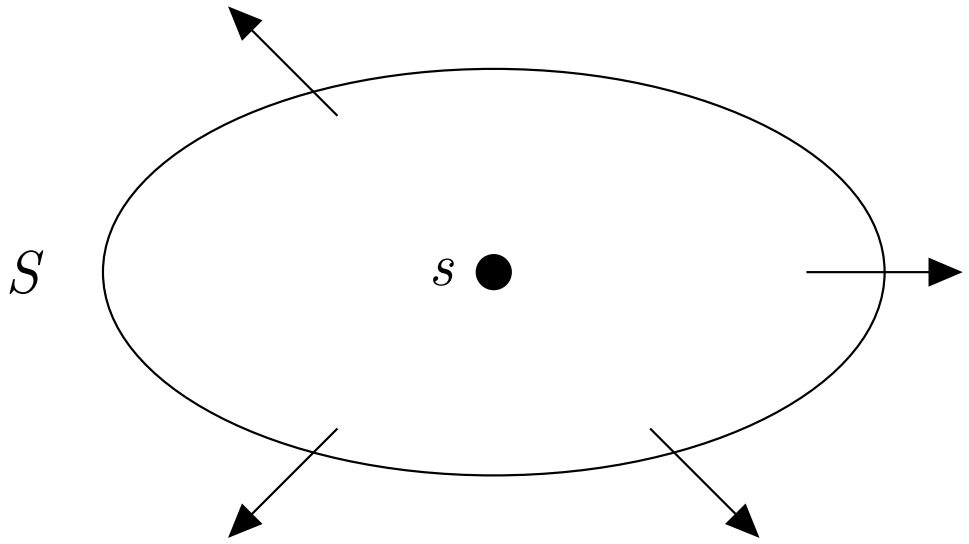
Algorithme de Dijkstra

<u>Idée</u> : Calculer les distances par ordre croissant depuis s.

Algorithme de Dijkstra

<u>Idée</u> : Calculer les distances par ordre croissant depuis s.

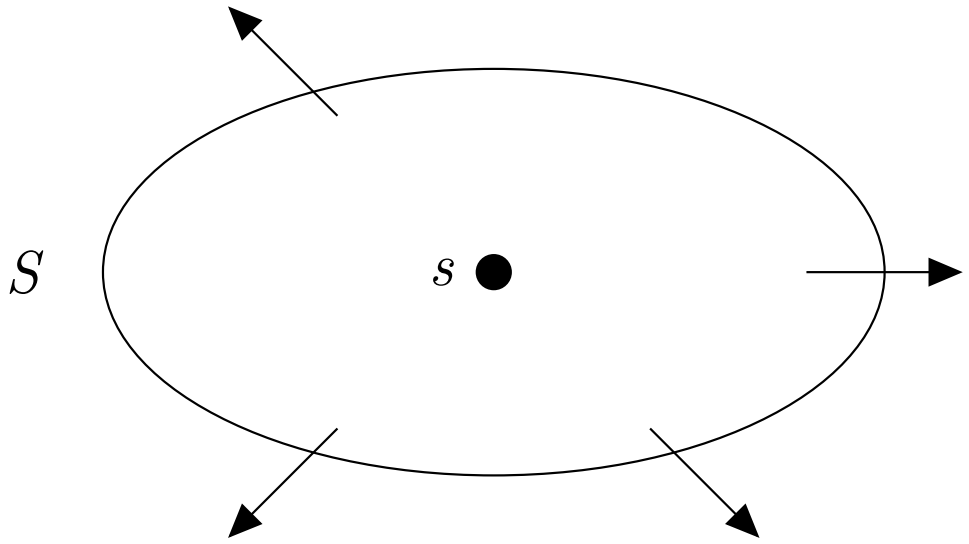
Soit $S \subset V$ l'ensemble des sommets de distance connue.



Algorithme de Dijkstra

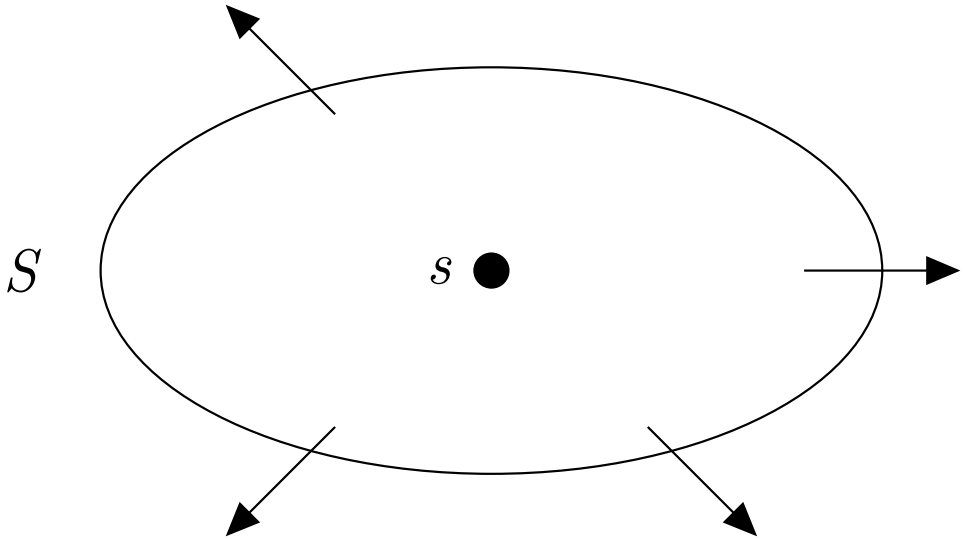
<u>Idée</u> : Calculer les distances par ordre croissant depuis s.

Soit $S \subset V$ l'ensemble des sommets de distance connue.



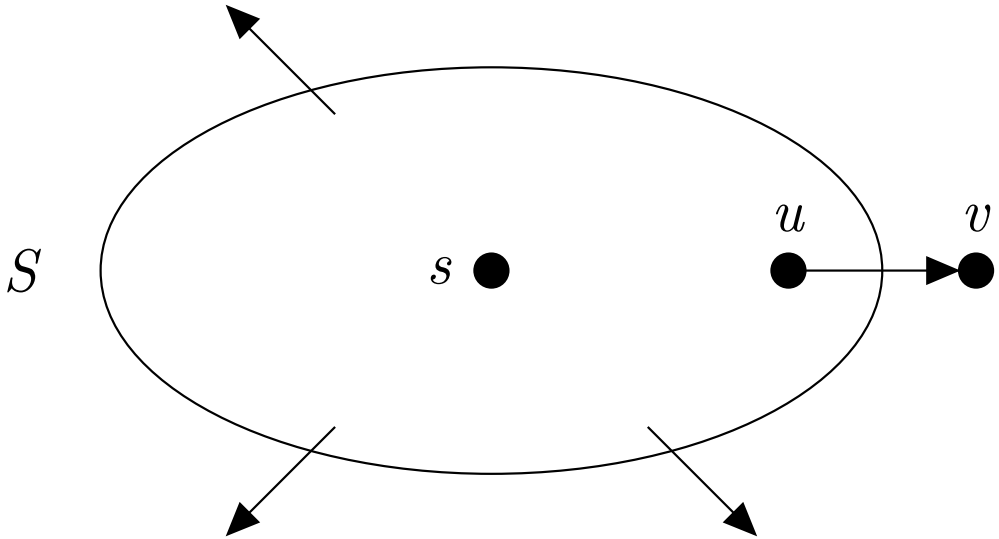
À chaque étape, on déduit la distance à un sommet de plus (et on l'ajoute à S).

Algorithme de Dijkstra



Soit $(u, v) \in \vec{E}$ tel que $v \notin S$ et $d(s, u) + w(u, v)$ est minimum.

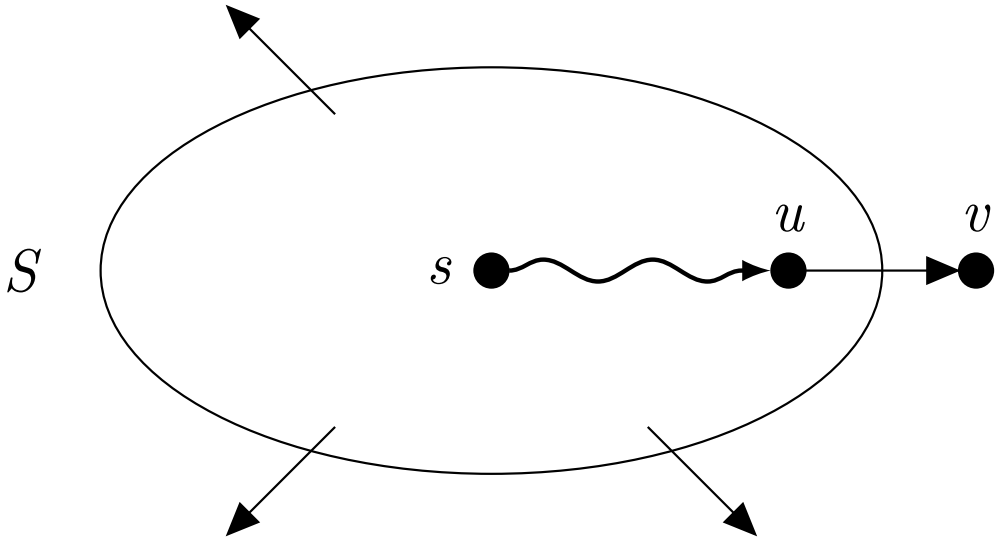
Algorithme de Dijkstra



Soit $(u, v) \in \vec{E}$ tel que $v \notin S$ et $d(s, u) + w(u, v)$ est minimum.
Alors :

$$\boxed{d(s, v) = d(s, u) + w(u, v)}$$

Algorithme de Dijkstra



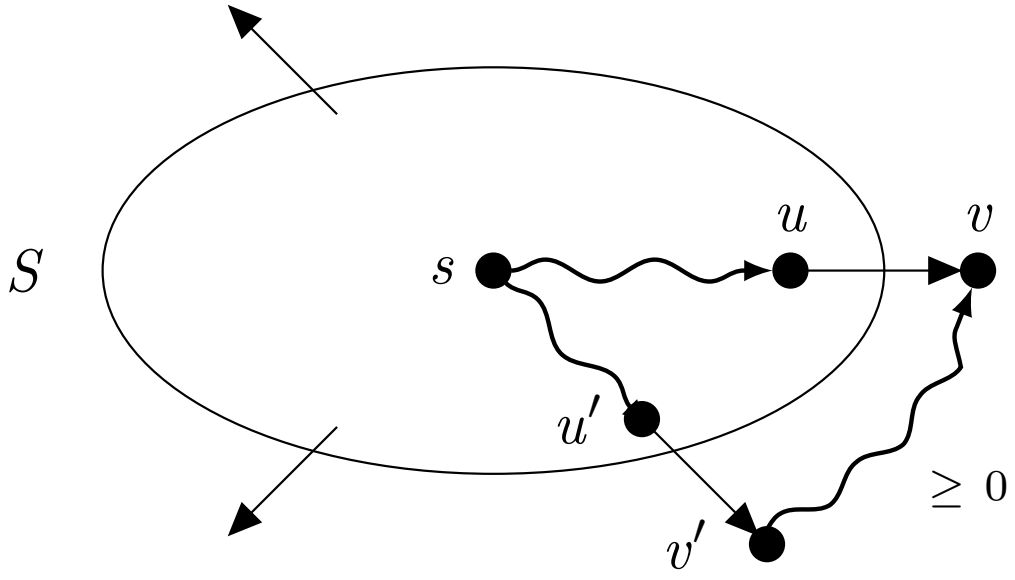
Soit $(u, v) \in \vec{E}$ tel que $v \notin S$ et $d(s, u) + w(u, v)$ est minimum.
Alors :

$$\boxed{d(s, v) = d(s, u) + w(u, v)}$$

<u>Preuve</u> :

1. Il existe un chemin de longueur $d(s, u) + w(u, v)$.

Algorithme de Dijkstra



Soit $(u, v) \in \vec{E}$ tel que $v \notin S$ et $d(s, u) + w(u, v)$ est minimum.
Alors :

$$\boxed{d(s, v) = d(s, u) + w(u, v)}$$

<u>Preuve</u> :

2. Un chemin C de s à v doit sortir de S avec un arc (u', v'). Comme les poids sont ≥ 0 :

$$\text{poids}(C) \geq d(s, u') + w(u', v') \geq d(s, u) + w(u, v)$$

Algorithme de Dijkstra

On stocke les sommets restants à visiter dans `q` ($= \overline{S}$) et on conserve un tableau `dist` des distances estimées tel que :

1. $\forall v \notin$ `q` : `dist[v]` $= d(s, v)$.
2. $\forall v \in$ `q` : `dist[v]` $= \min\limits_{u \notin q} d(s, u) + w(u, v)$.

Algorithme de Dijkstra

On stocke les sommets restants à visiter dans q ($= \overline{S}$) et on conserve un tableau dist des distances estimées tel que :

1. $\forall v \notin$ q : dist[v] $= d(s, v)$.
2. $\forall v \in$ q : dist[v] $= \min_{u \notin q} d(s, u) + w(u, v)$.

Algorithme de Dijkstra :

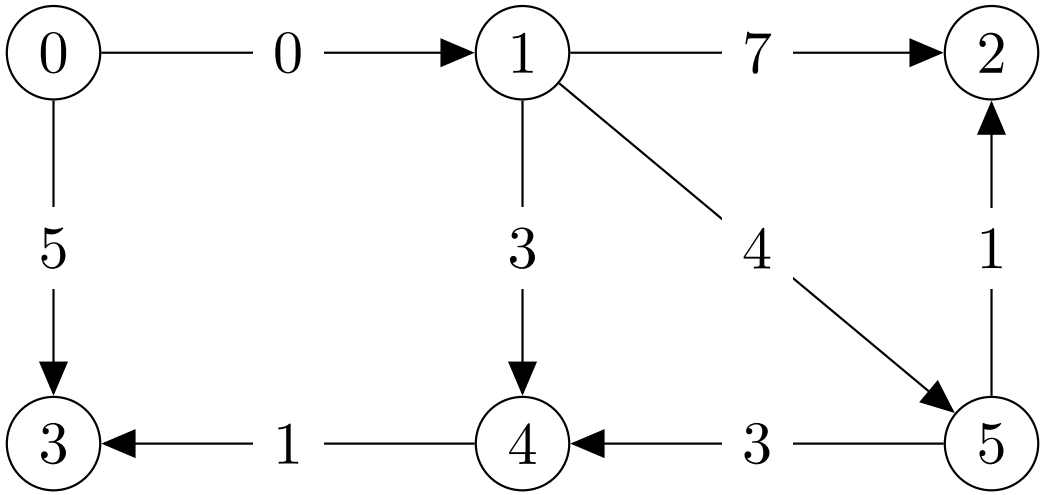
```
Initialement : q contient tous les sommets
               dist[s] = 0
               dist[v] = float("inf"), si v != s

Tant que q est non vide:
    Extraire u de q tel que dist[u] soit minimum
    Pour tout voisin v de u:
        Si dist[u] + w(u, v) < dist[v]:
            dist[v] = dist[u] + w(u, v)
```

Algorithme de Dijkstra : Exemple

Exercice

Appliquer l'algorithme de Dijkstra depuis $s = 0$ sur le graphe suivant, en mettant `dist[v]` à côté de chaque sommet `v` :



Algorithme de Dijkstra : Avec une file de priorité

Initialement : `next` contient tous les sommets

`dist.(r) <- 0` et `dist.(v) <-` ∞, $\forall$ `v` $\neq$ `r`

Tant que `next` $\neq \emptyset$:

Extraire `u` de `next` tel que `dist.(u)` soit minimum

Pour tout voisin `v` de `u` :

```
dist.(v) <- min dist.(v) (dist.(u) + w u v)
```

Complexité si `next` est une **file de priorité min** :

Algorithme de Dijkstra : Avec une file de priorité

Initialement : `next` contient tous les sommets

`dist.(r) <- 0` et `dist.(v) <-` ∞, $\forall$ `v` $\neq$ `r`

Tant que `next` $\neq \emptyset$:

Extraire `u` de `next` tel que `dist.(u)` soit minimum

Pour tout voisin `v` de `u` :

```
dist.(v) <- min dist.(v) (dist.(u) + w u v)
```

Complexité si `next` est une **file de priorité min** :

1. n extractions du minimum $\longrightarrow$ $\mathrm{O}(n\log(n))$
2. au plus p mises à jour $\longrightarrow$ $\mathrm{O}(p\log(n))$

Total : $\mathrm{O}(n\log(n)) + \mathrm{O}(p\log(n)) = \boxed{\mathrm{O}(p\log(n))}$.

Algorithme de Dijkstra : Avec une file de priorité

On suppose avoir un module de file de priorité min avec les fonctions suivantes :

```
(* PriorityQueue.make () renvoie une file de priorité vide *)
PriorityQueue.make : unit -> 'a priority_queue

(* PriorityQueue.add q e p ajout e avec priorité p dans q *)
PriorityQueue.add : 'a priority_queue -> 'a -> int -> unit

(* PriorityQueue.empty q détermine si q est vide *)
PriorityQueue.empty : 'a priority_queue -> bool

(* PriorityQueue.take_min q extrait l'élément de q de priorité min
PriorityQueue.take_min : 'a priority_queue -> 'a

(* PriorityQueue.update q e p met à jour la priorité de e dans q *
PriorityQueue.take_min : 'a priority_queue -> 'a
```

Algorithme de Dijkstra : Avec une file de priorité

```
let dijkstra g s =
    let n = Array.length g in
    let dist = Array.make n max_int in
    dist.(s) <- 0;
    let q = PriorityQueue.make () in
    for v = 0 to n - 1 do
        PriorityQueue.add v dist.(v)
    done;
    while not PriorityQueue.empty q do
        let u = PriorityQueue.take_min q in
        for v = 0
            d = dist[u] + G[u][v]
            if v in q and d < dist[v]:
                q.update(v, d)
                dist[v] = d
    return dist
```

Algorithme de Dijkstra : Avec une file de priorité

Problème : la fonction de mise à jour d'un élément dans un tas demande de connaître l'indice de l'élément à modifier.

Il faudrait maintenir un tableau qui donne l'indice (dans le tas) d'un sommet. C'est fastidieux ...

On pourrait utiliser un ABR équilibré : pour mettre à jour il suffit d'appeler `del` puis `add`.

Algorithme de Dijkstra : Avec une file de priorité

Ma solution « personnelle » plus simple : ajouter des couples (distance estimée de `v`, `v`) sans jamais mettre à jour les éléments de la FP. On peut donc avoir plusieurs fois le même sommet dans la FP :

Algorithme de Dijkstra : Avec une file de priorité

Ma solution « personnelle » plus simple : ajouter des couples (distance estimée de `v`, `v`) sans jamais mettre à jour les éléments de la FP. On peut donc avoir plusieurs fois le même sommet dans la FP :

Initialement : `q` contient `(0, r)`
`dist.(r) <- 0` et `dist.(v) <-` ∞, $\forall$ `v` $\neq$ `r`

Tant que `q` $\neq \emptyset$:
    Extraire `(d, u)` de `q` tel que `d` soit minimum
    Si `dist.(u)` $= \infty$:
        `dist.(u) <- d`
        Pour tout voisin `v` de `u` :
            Ajouter `(d + w u v, v)` à `q`

Algorithme de Dijkstra : Avec une file de priorité

Ma solution « personnelle » plus simple : ajouter des couples (distance estimée de v, v) sans jamais mettre à jour les éléments de la FP. On peut donc avoir plusieurs fois le même sommet dans la FP.

```
let dijkstra g w r =
  let n = Array.length g in
  let q = empty () in (* file de priorité vide *)
  let dist = Array.make n max_int in (* dist.(v) va être d(r, v)
  add q (0, r);
  while not (is_empty q) do
    let d, u = take_min q in
    if dist.(u) = max_int then (
        dist.(u) <- d;
        List.iter (fun v -> add q (v, d + w u v)) g.(u)
    )
  done;
  dist
```

Plus courts chemins avec Dijkstra

Initialement : `next` contient tous les sommets

`dist.(r) <- 0` et `dist.(v) <-` ∞, $\forall$ `v` $\neq$ `r`

Tant que `next` $\neq \emptyset$:

Extraire `u` de `next` tel que `dist.(u)` soit minimum

Pour tout voisin `v` de `u` :

```
dist.(v) <- min dist.(v) (dist.(u) + w u v)
```

Question

Comment modifier l'algorithme pour connaître les plus courts chemins?

Plus courts chemins avec Dijkstra

Tant que `next` $\neq \emptyset$:

```
    Extraire u de next tel que dist.(u) soit minimum
    Pour tout voisin v de u :
        Si dist.(v) > dist.(u) + w u v :
            dist.(v) <- dist.(u) + w u v
            pere.(v) <- u
```

On peut conserver dans `pere.(v)` le prédécesseur de `v` dans un plus court chemin de `r` à `v`.
`v`, `pere.(v)`, `pere.(pere.(v))`, ... jusqu'à `r` donne un chemin (à l'envers) de `r` à `v`.

Plus courts chemins avec Dijkstra

Tant que `next` $\neq \emptyset$:

```
    Extraire u de next tel que dist.(u) soit minimum
    Pour tout voisin v de u :
        Si dist.(v) > dist.(u) + w u v :
            dist.(v) <- dist.(u) + w u v
            pere.(v) <- u
```

On peut conserver dans `pere.(v)` le prédécesseur de `v` dans un plus court chemin de `r` à `v`.
`v`, `pere.(v)`, `pere.(pere.(v))`, ... jusqu'à `r` donne un chemin (à l'envers) de `r` à `v`.

Le graphe des pères est un arbre (**un arbre des plus courts chemins**).

Plus courts chemins avec Dijkstra

On stocke des triplets (distance estimé de v, v, père de v) dans la FP :

```
let dijkstra g w r =
  let n = Array.length g in
  let q = empty () in
  let pere = Array.make n (-1) in
  add q (0, r, r);
  while not (is_empty q) do
    let d, u, p = take_min q in
    if pere.(u) = max_int then (
      pere.(u) <- p;
      List.iter (fun v -> add q (sum d (w u v)), v, u) g.(u)
    )
  done;
  pere
```

(On n'a plus besoin de dist)